Project 2: Fourier Transform

Niño Aloysius V. De Mesa

ENGG 151.01-A

March 15, 2026

Basic Test for Program:

To verify that the program is running as intended, the output of the actual application would be compared with the basic test data provided by the specifications of the project. Figure 1 shows the content of the x.signal file to be used while Figure 2 shows the expected resulting DFT after performing the command line function: “dft x.signal 32 4 8 8”.

Basic Test

signal file

x.signal

0	-0.130733614
	-1.238039900
	-1.440074782
	-0.589107641
	0.692622920
	1.467898298
	1.169826725
	0.016362137
	-1.149066665
	-1.474282487
	-0.721483153
	0.558874353
	1.430575426
	1.256220532
	0.163300312
	-1.049027290
	-1.494292047
	-0.846910389
	0.419743521
	1.379475331
	1.330516250
	0.308665816
	-0.938885208
	-1.499910758
	-0.964181415
	0.276570329
	1.315090134
	1.391998368
	0.451058699
	-0.819701149
	-1.491084507
	-1.072166846

Figure 1. Basic test data showing the provided values of the x.signal file.

resulting DFT

frequency (Hz)	real part	imaginary part
4	-16.016414	-1.271327
4.5	-2.091738	23.908673
5	14.523652	1.379051
5.5	0.000000	0.000000
6	4.458294	0.479847
6.5	0.000000	0.000000
7	2.498667	0.291875
7.5	0.000000	0.000000
8	1.690067	0.206077

frequency (Hz)	magnitude	phase (degrees)
4	16.066792	-175.461577
4.5	24.000000	95.000000
5	14.588977	5.424093
5.5	0.000000	-62.499637
6	4.484042	6.143102
6.5	0.000000	-0.912811
7	2.515656	6.662659
7.5	0.000000	7.360520
8	1.702584	6.951986

Figure 2. Basic test data showing the expected resulting DFT

Moving on, Figure 3 then shows the contents of a signal file named x.signal with the provided values as seen on Figure 1.

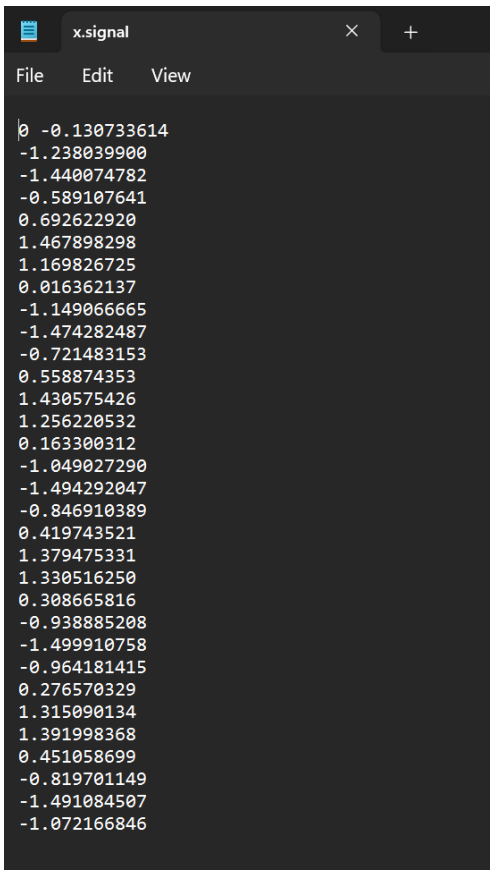


Figure 3. Content of the “x.signal” file

Afterwards, to execute the program, the specifications require it to run in the console of the computer. Figure 4 first shows the version of the compiler used to execute the program: **g++.exe (MinGW.org GCC-6.3.0-1) 6.3.0**, while Figure 5 shows the version and build of the operating system of the computer: **Windows 11 Home Single Language, 25H2, build 26100.7840**. Figure 6 then shows the version of the IDE used for the project: **Code::Blocks 25.03**

.

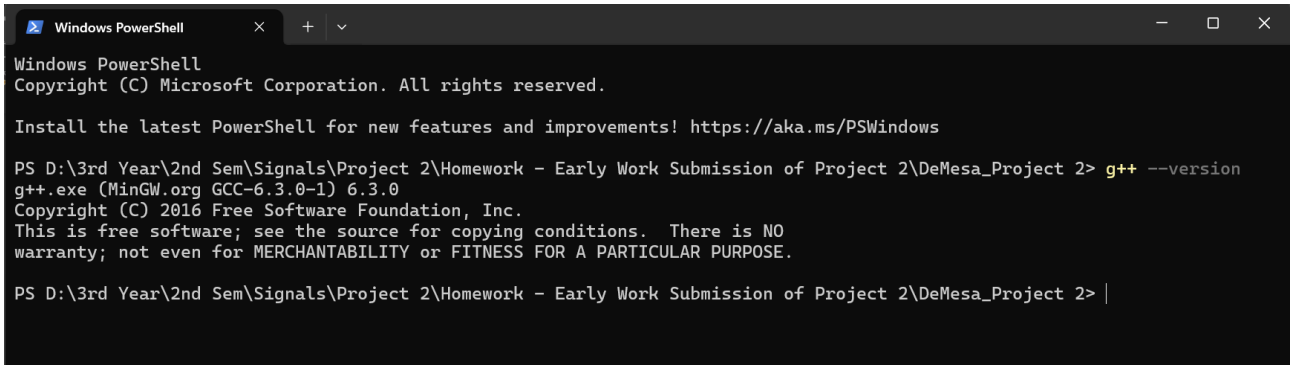


Figure 4: Compiler - g++.exe (MinGW.org GCC-6.3.0-1) 6.3.0

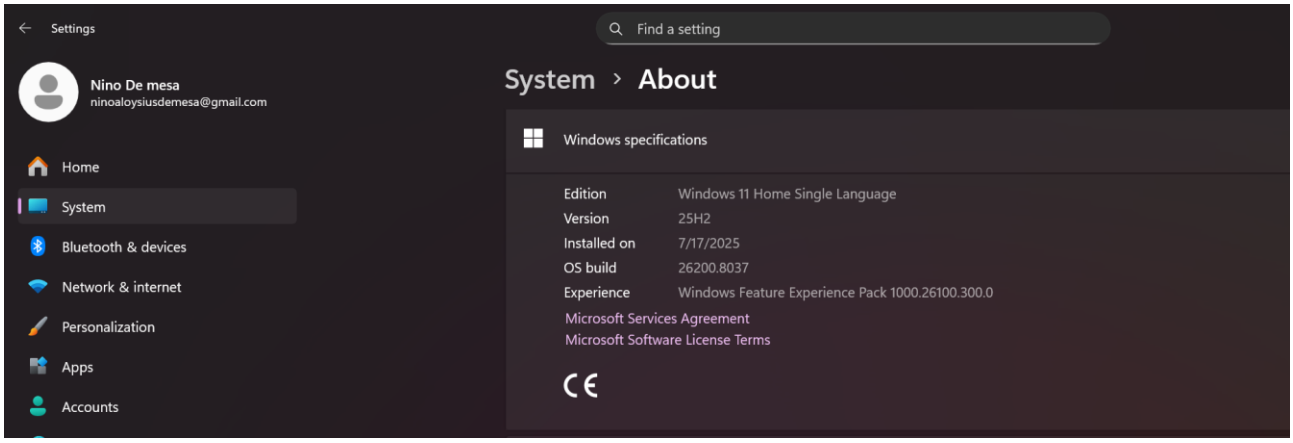


Figure 5: operating system - Windows 11 Home Single Language, 25H2, build 26100.7840

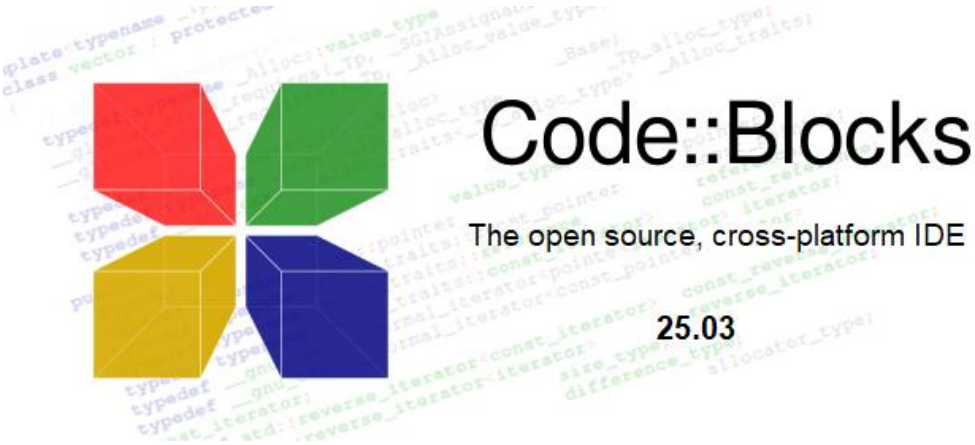


Figure 6: IDE -Code::Blocks 25.03

To test the results, Figure 7 shows the program being compiled in the IDE while Figure 8 shows it being run and executed through the console of the computer.

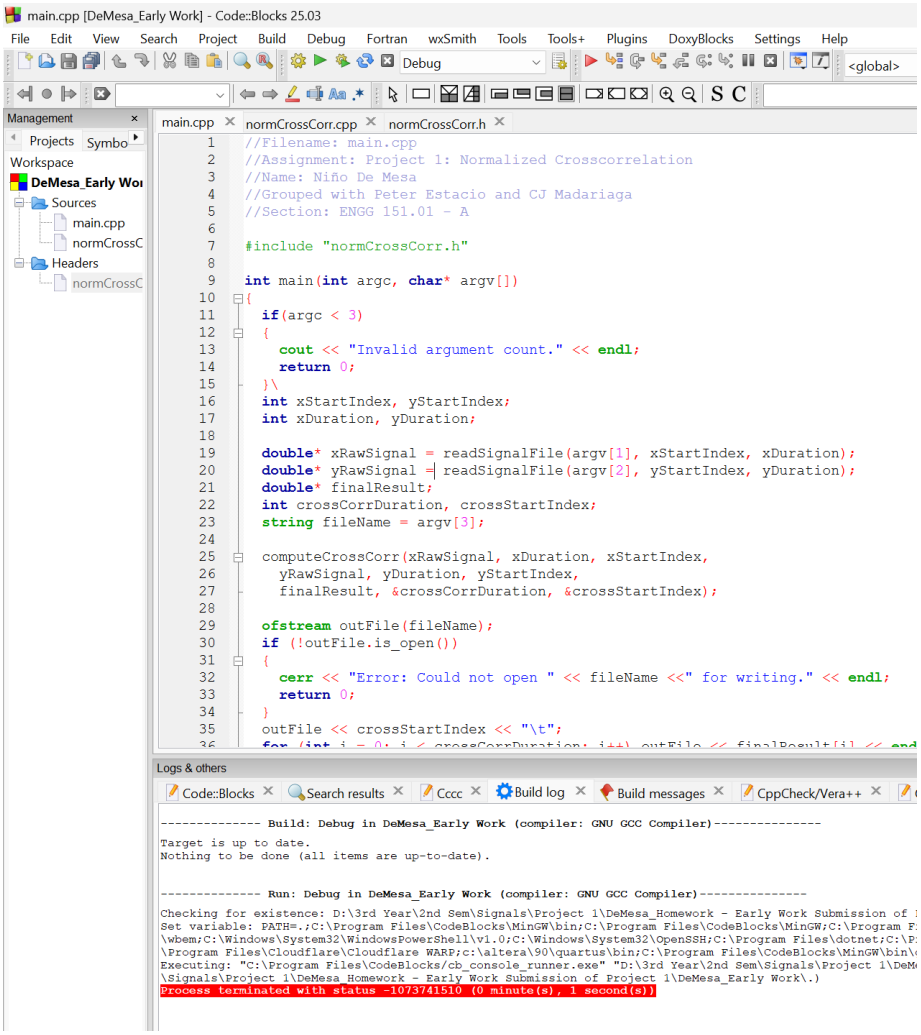


Figure 7: sample screenshot - compilation on Code::Blocks

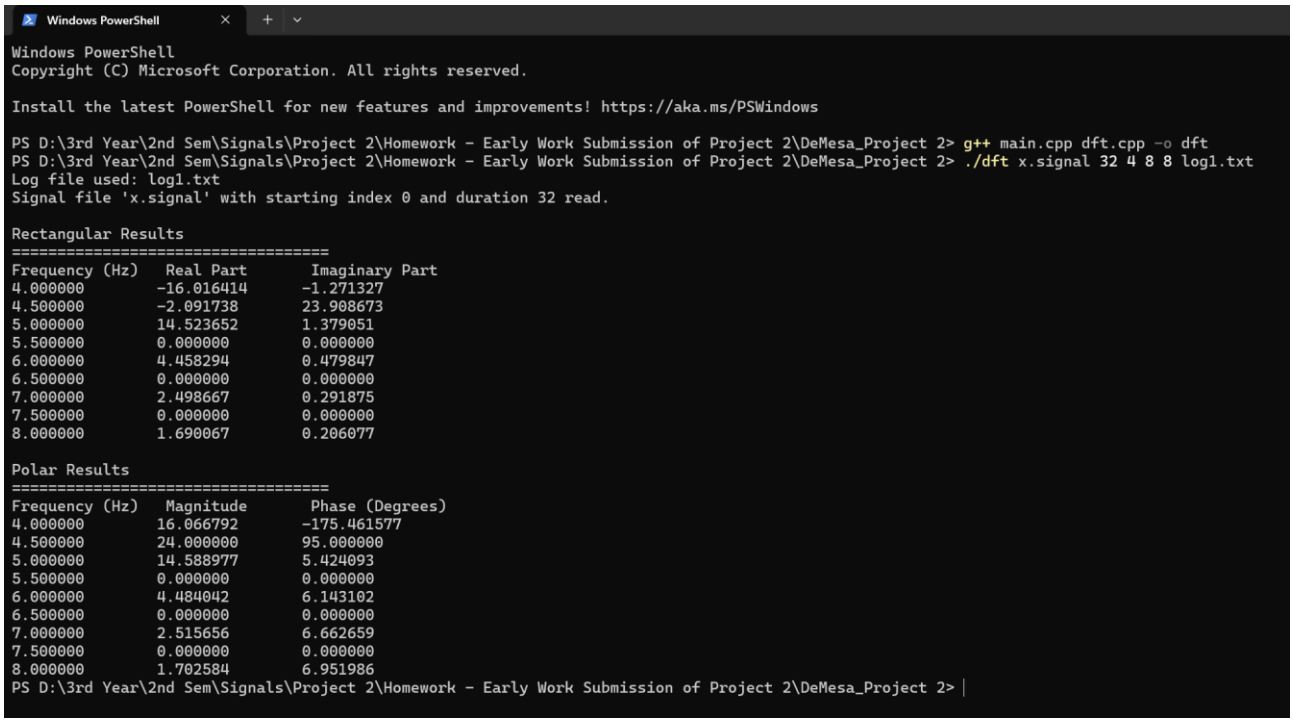
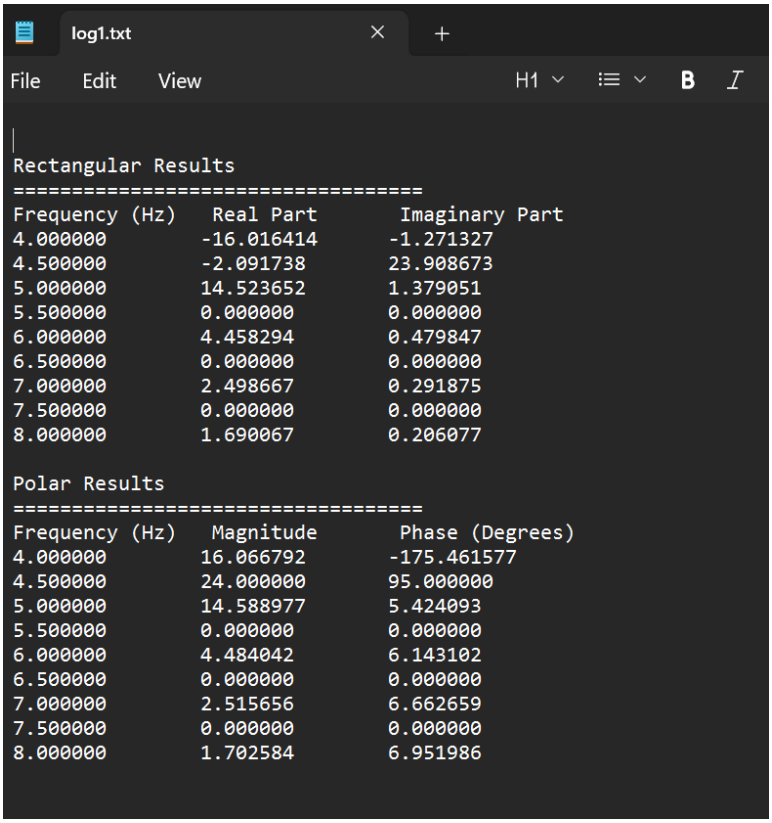


Figure 8: sample screenshot – executable dft.exe typing its name with all of the required paramters and files for the signal at the prompt from a path with name of author (DeMesa)



```
Rectangular Results
===================================
Frequency (Hz)   Real Part       Imaginary Part
4.000000        -16.016414      -1.271327
4.500000        -2.091738       23.908673
5.000000        14.523652       1.379051
5.500000        0.000000        0.000000
6.000000        4.458294        0.479847
6.500000        0.000000        0.000000
7.000000        2.498667        0.291875
7.500000        0.000000        0.000000
8.000000        1.690067        0.206077

Polar Results
===================================
Frequency (Hz)   Magnitude       Phase (Degrees)
4.000000        16.066792       -175.461577
4.500000        24.000000       95.000000
5.000000        14.588977       5.424093
5.500000        0.000000        0.000000
6.000000        4.484042        6.143102
6.500000        0.000000        0.000000
7.000000        2.515656        6.662659
7.500000        0.000000        0.000000
8.000000        1.702584        6.951986
```

Figure 9: Content of the output “log.txt” file which has the resulting DFT values

Since the number of steps in the basic test provided is less than 10, the resulting DFT output that was sent to the log file would also be shown in the console as observed in Figure 8. Additionally, it is also seen that the resulting DFT values were also like one that was expected based on the given basic test specification as seen in Figure 2. Finally, Figure 9 then shows the output sent to the log file named “log1.txt” which also produced the same results from Figures 2 and 8 showing that the basic test was done successfully.

Code locator:

code items	file name	line numbers
DFT function	dft.cpp	112-143
declaration of double * that are passed to the DFT function	main.cpp	42-46
separate processing of first line of signal file	dft.cpp	60-83
input validation of integers in processing signal files	dft.cpp	9-24
input validation of floating point numbers in processing signal files	dft.cpp	26-41

Project Evaluation - Fourier Transform		
Item	Points	Rubrics
early work	8/8	(all or nothing) 8: at most 10% of the code in the final implementation differs from that in early work submission
implementation and testing environment report	4/4	4 - all instructions followed correctly
basic test	50/50	50 - correctly computes the DFT of basic test data
	6/6	6 - screenshot submitted showing the correctly computed DFT of basic test data as it appears on the console output
generality of DFT	6/6	6 - the DFT is computed correctly and efficiently at all times
DFT function	6/6	6 - the DFT is computed correctly and efficiently at all times, by a function as specified
processing of signal files	4/4	4 - The file is opened only once and the first line of the signal file is processed correctly and separately from code processing the rest of the signal file
input validation	4/4	4 - integers and floating point numbers are fully validated when signal files are processed
command line	4/4	4 - application processes command line arguments correctly at all times
application feedback	4/4	4 - application produces required outputs and provides appropriate feedback at all times, as specified
self-evaluation	4/4	4 - self-evaluation accurate (or evaluating this item leads to an error)
total	100	